method being tested is also coupled to those internal details. That is bad, not just if the third-party tool changes, but also if you want to refactor your code; the dependency will make that change more complicated.

The solution, in most cases, is to create a method or class in your application that calls the third-party tool and stubs that method (while also writing tests to ensure that the wrapper does the right thing). You'll see a larger example of this technique in Chapter 14, *Testing External Services*, on page 285. However, in a smaller case you can do this easily as follows:

```
class Project
  def self.create_from_controller(params)
    create(params)
  end
end
```

And then in a test:

```
it "creates a project" do
  allow(Project).to receive(:create_from_controller).and_return(Project.new)
end
```

I admit that, oversimplified like this, the technique seems like overkill. (All object-oriented techniques seem like overkill until you suddenly realize you needed them six months ago.) If you create a method like this wrapper, however, you'll often find that the functionality is shared in multiple places, giving you one point of contact with the third-party tool rather than several. You'll also find that these methods attract what would otherwise be duplicated behavior. Both of these are good things.

When to Mock, When to Stub

If you're using your test doubles to take the place of real objects that are hard or impossible to create in a test environment, it's probably a good idea to use stubs rather than mocks. If you're using the fake value as an input to a different process, then you should test that process directly using the fake value rather than a mock. Adding the mock expectation just gives you another thing that can break, which in this use case is probably not related to what you're testing.

On the other hand, if you're testing the relationship between different systems of your code, you should tend to use mocks to verify the behavior of one part of the code as it calls the other.

Mocks are particularly good at testing across boundaries between subsystems—for example, controller testing to isolate the controller test from the